ARC-AGI + LLM + RL Reference Document

> Comprehensive reference for an LLM + RL project targeting the ARC-AGI benchmark.

1. ARC Dataset Overview

The **Abstraction and Reasoning Corpus (ARC)** was created by Francois Chollet in 2019, introduced in the paper "*On the Measure of Intelligence*" (arxiv.org/abs/1911.01547). It is designed to measure general fluid intelligence — the ability to acquire new skills on-the-fly from minimal examples — rather than memorized pattern matching.

Task Structure

- Each task is a JSON file containing:
  - `train` : 2–10 demonstration input/output pairs
  - `test` : 1–3 test input/output pairs (output withheld during evaluation)
- **Grids**: Rectangular matrices of integers 0–9, ranging from 1x1 to 30x30
- **Colors**: Each integer maps to a color:

Value	Color
0	Black
1	Blue
2	Red
3	Green
4	Yellow
5	Gray
6	Magenta
7	Orange

Value	Color
8	Azure
9	Maroon

Core Knowledge Priors

ARC tasks are designed to require only priors shared by all humans:

- **Objectness**: Contiguous same-color regions form objects; objects persist, can move/transform
- **Goal-directedness**: Transformations have a purpose/pattern
- **Numbers & counting**: Small integer quantities, sorting, one-to-one correspondence
- **Basic geometry & topology**: Lines, rectangles, symmetry, rotation, reflection, scaling, containment, connectivity

Evaluation

- Output must be **pixel-perfect** — every cell must exactly match the expected output
- No partial credit; the entire grid must be correct

2. Task Examples

Common Transformation Types

- **Color replacement**: Map one color to another based on a rule
- **Symmetry completion**: Fill in missing parts to complete a symmetric pattern
- **Object gravity**: Move objects in a direction until they hit a boundary or another object
- **Pattern scaling**: Enlarge or shrink a pattern by a factor
- **Flood fill**: Fill connected regions with a specific color
- **Counting & encoding**: Count objects and encode the count as grid cells

Concrete JSON Example

```
{
  "train": [
    {
      "input":  [[0, 0, 0], [0, 1, 0], [0, 0, 0]],
      "output": [[1, 1, 1], [1, 1, 1], [1, 1, 1]]
    },
    {
      "input":  [[0, 0, 0, 0], [0, 0, 2, 0], [0, 0, 0, 0], [0, 0, 0, 0]],
      "output": [[2, 2, 2, 2], [2, 2, 2, 2], [2, 2, 2, 2], [2, 2, 2, 2]]
    }
  ],
  "test": [
    {
      "input":  [[0, 0, 0], [0, 0, 0], [0, 3, 0]],
      "output": [[3, 3, 3], [3, 3, 3], [3, 3, 3]]
    }
  ]
}
```

Rule: Find the single non-black cell, then fill the entire grid with that color.

3. Data Sources

Source	URL
ARC-AGI-1 GitHub (400 train + 400 eval)	https://github.com/fchollet/ARC-AGI
ARC-AGI-2 GitHub (1000 train + 120 eval)	https://github.com/arcprize/ARC-AGI-2
Kaggle ARC Prize 2025	https://www.kaggle.com/competitions/arc-prize-2025
HuggingFace v1 public eval	https://huggingface.co/datasets/arcprize/arc_agi_v1_public_eval
HuggingFace v2 public eval	https://huggingface.co/datasets/arcprize/arc_agi_v2_public_eval
Try ARC in browser	https://arcprize.org/play

Source	URL
RE-ARC (synthetic data generator)	https://github.com/michaelhodel/re-arc

> **WARNING**: The dataset `allenai/ai2_arc` on HuggingFace is a **DIFFERENT** dataset — it contains multiple-choice science questions (AI2 Reasoning Challenge), not visual grid puzzles. Do not confuse the two.

4. Leaderboard & Competition

- **Website**: https://arcprize.org/
- **Leaderboard**: https://arcprize.org/leaderboard
- **Grand Prize**: $700,000 for achieving 85% on the ARC-AGI-2 private evaluation set (currently unclaimed)

ARC-AGI-1 Top Scores

Rank	Team/Method	Score
1	OpenAI o3-high	87.5%
2	MindsAI	55.5%
3	ARChitects	53.5%
4	Ryan Greenblatt	50%

ARC-AGI-2 Top Scores

Rank	Team/Method	Score
1	NVARC	24%
2	ARChitects	16.5%
3	MindsAI	12.6%

Human Baselines

- ARC-AGI-1: ~85% average
- ARC-AGI-2: ~60% average

5. ARC-AGI-1 vs ARC-AGI-2

Aspect	AGI-1	AGI-2
Training tasks	400	1,000
Public evaluation tasks	400	120
Attempts per test input	3	2
Best AI score	87.5% (o3-high)	24% (NVARC)
Human average	~85%	~60%

What AGI-2 Adds

- **Symbolic interpretation**: Tasks requiring understanding of abstract symbols
- **Compositional reasoning**: Multiple transformation rules chained together
- **Contextual rules**: Rules that depend on context (e.g., different behavior for different objects)
- **Brute-force resistance**: Tasks designed to resist enumeration-based approaches
- **Efficiency metric**: Penalizes solutions that use excessive compute

6. Top Solution Approaches

A. Code Generation (Induction) — e.g., Greenblatt

The LLM generates Python programs that implement the input-to-output transformation.

Pipeline (Greenblatt's approach, 50% on AGI-1):

1. Convert each task grid into **5 complementary text representations** (nested lists, row-by-row, color names, spreadsheet coords, connected components)

2. Sample ~**8,000 candidate programs** from GPT-4o across all representations
3. **Filter**: Execute each program on training inputs; keep only those producing correct training outputs
4. **Revision phase**: Feed failing programs + error messages back to the LLM for debugging
5. **Majority vote** among surviving programs on test input

Key insight: Using multiple complementary representations reduces required compute by ~10x compared to a single representation.

- **Repo**: https://github.com/rgreenblatt/arc_draw_more_samples_pub
- **Blog**: https://blog.redwoodresearch.org/p/getting-50-sota-on-arc-agi-with-gpt

B. Test-Time Training (Transduction) — e.g., MindsAI, ARChitects

Fine-tune a pretrained model on each individual task at test time.

Pipeline:

1. **Data augmentation**: Apply transformations to the 2–10 demo pairs:
   - Rotations (0/90/180/270)
   - Reflections (horizontal/vertical)
   - Color permutations
   - → Produces 100–500+ augmented examples per task
2. **Fine-tune**: Apply 64–128 LoRA gradient steps on the augmented examples
3. **Predict**: The fine-tuned model directly outputs the grid (no intermediate code)
4. Can achieve up to **6x improvement** over frozen baselines

Key advantage: Works well for tasks where the pattern is hard to express as code but easy to learn from examples.

C. Hybrid Approaches

The best competition scores typically combine both induction and transduction:

- Run code generation and test-time training in parallel
- Aggregate predictions via voting or confidence scoring

D. OpenAI o3 Approach

- **Method**: Search over chains of thought (natural-language "programs") with backtracking
- **Compute**: 172x more compute than standard inference; approximately **$4,560 per task** at high compute
- **Results**: 87.5% on AGI-1 but only ~4% on AGI-2 (suggesting memorization/overfitting to AGI-1 patterns)

7. Vision vs Text Input

Vision Models Fail on ARC

- GPT-4V (vision) "produced no meaningful results" on ARC tasks
- Grid images are too abstract for vision encoders trained on natural images
- **Text-based representations dominate** all successful approaches

Text Representation Formats

Format	Example (3x3 grid)
Nested lists	`[[0, 1, 0], [1, 0, 1], [0, 1, 0]]`
Row-by-row	`Row 0: 0 1 0\nRow 1: 1 0 1\nRow 2: 0 1 0`
Color names	`Row 0: black blue black\nRow 1: blue black blue`
Spreadsheet coords	`A1=black, B1=blue, C1=black, A2=blue, ...`
Connected components	`Object 1 (blue): cells (0,1),(1,0),(1,2),(2,1)`
Input-output diff	`Changed: (0,0) from black to red, ...`

BPE Tokenization Problem

Standard BPE tokenizers merge multi-digit numbers into single tokens (e.g., `12` becomes one token), but in ARC grids, `1` and `2` are different colors and must be treated as separate values.

Solutions:

- Use color names instead of digits ("blue" instead of "1")

- Use single-character symbols with guaranteed separate tokenization
- Custom tokenizers that never merge digit pairs
- Separator characters between digits (e.g., `1,2,0,1`)

Greenblatt's Key Insight

Using **multiple complementary representations** of the same task and sampling programs across all of them reduces required compute by approximately **10x** compared to using any single representation.

8. RL Approaches for ARC

Existing Work

DreamerV3 on ARC (2024)

- Model-based RL applied directly to grid manipulation
- 5-operation action space (place color, copy, etc.)
- Only tested on 4 simple tasks — does not scale
- Demonstrates feasibility but impractical for the full benchmark

SOAR (ICML 2025)

- Evolutionary program synthesis + fine-tune LLM on its own successful search traces
- Achieves 52% on ARC-AGI-1
- Key idea: Self-improvement loop where the model learns from its own discoveries
- Paper: arxiv.org/abs/2507.14172

Reasoning Gym (2025)

- GRPO-based RLVR (Reinforcement Learning from Verifiable Rewards) framework
- Includes ARC tasks as one of several reasoning benchmarks
- Open-source infrastructure for training reasoning models

DeepSeek-R1 (Nature 2025)

- Pure RL with GRPO produces emergent chain-of-thought reasoning
- Not ARC-specific but demonstrates that RL alone can elicit complex reasoning in LLMs
- Key finding: No SFT warmup needed — reasoning emerges from reward signal alone

Why ARC Is Ideal for RL

1. **Deterministic exact-match verification** = perfect reward oracle (no reward hacking possible)
2. **Binary reward**: Grid match (correct/incorrect) — unambiguous signal
3. **Graded reward** option: Cell-level accuracy (fraction of cells correct) for denser signal
4. **Multi-level feedback** hierarchy:
   - Syntax error in generated code → lowest reward
   - Runtime error (code runs but crashes) → low reward
   - Wrong output (code runs, produces incorrect grid) → medium reward
   - Correct output → maximum reward
5. **Small output space**: Grids are at most 30x30 with 10 possible values — tractable
6. **No human labeling needed**: Verification is fully automated

Key RL Algorithms for LLM + Code Generation

Algorithm	Used In	Key Feature
PPO	CodeRL, RLEF, StepCoder	Standard policy gradient with clipping; stable training
GRPO	DeepSeek-R1, Reasoning Gym	Group relative policy optimization; no critic network needed
DPO	Various	Direct preference optimization; simpler than PPO, offline
RRHF	Various	Rank responses by reward, align via ranking loss

CodeRL (NeurIPS 2022): RL for code generation with unit test feedback

RLEF (ICML 2025): Multi-turn refinement — LLM generates code, executes it, sees errors, revises; RL optimizes the full multi-turn trajectory

StepCoder (ACL 2024): Curriculum RL for code generation; decomposes long code into steps

Test-Time Search Strategies

MCTS over Thoughts/Programs

- **ReST-MCTS*** (NeurIPS 2024): Monte Carlo Tree Search over reasoning steps, using a process reward model to guide search
- **RethinkMCTS**: Builds search tree over partial solutions, backtracking on failures

Evolutionary Search

- **Berman (2024)**: Evolutionary program synthesis achieves 79.6% on AGI-1 at $8.42/task
- Mutate and crossover Python programs; select by training-set accuracy

Multi-Turn Refinement with Execution Feedback

- Generate code → execute → observe error/wrong output → revise → repeat
- RLEF optimizes this loop end-to-end with RL
- Natural fit for ARC: immediate, deterministic feedback after each attempt

Open Research Gap

> **No paper yet combines end-to-end RL training of an LLM for ARC program synthesis with MCTS test-time search.** This represents a significant opportunity: train the LLM's policy with RL (using execution feedback as reward), then deploy MCTS at test time to search over the trained policy's program space.

9. Key Papers & Repos

Papers

Paper	Venue/Year	Relevance
Chollet, "On the Measure of Intelligence"	arxiv.org/abs/1911.01547, 2019	Introduces ARC, defines intelligence measurement
Greenblatt, "Getting 50% on ARC-AGI with GPT-4o"	Blog, 2024	Code generation approach, multi-representation
TTT (Test-Time Training) for ARC	arxiv.org/abs/2411.07279, 2024	Per-task fine-tuning with augmentation
SOAR	arxiv.org/abs/2507.14172, ICML 2025	Evolutionary synthesis + self-improvement
RLEF	ICML 2025	Multi-turn RL with execution feedback

Paper	Venue/Year	Relevance
StepCoder	ACL 2024	Curriculum RL for code generation
ReST-MCTS*	NeurIPS 2024	MCTS over reasoning steps with PRM
DeepSeek-R1	Nature 2025	Pure RL emergent reasoning with GRPO
CodeRL	NeurIPS 2022	RL for code generation with unit tests

Repositories

Repo	Description
https://github.com/fchollet/ARC-AGI	Official ARC-AGI-1 dataset
https://github.com/arcprize/ARC-AGI-2	Official ARC-AGI-2 dataset
https://github.com/rgreenblatt/arc_draw_more_samples_pub	Greenblatt's code generation solution
https://github.com/michaelhodel/re-arc	RE-ARC synthetic data generator
https://github.com/open-thought/arc-agi-2	Open-source community solutions for AGI-2

10. Quick-Start Checklist

- ☐ Download ARC-AGI-1 training set (400 tasks) from GitHub
- ☐ Load and visualize tasks (use `matplotlib` with the color map above)
- ☐ Implement a baseline: prompt an LLM to generate Python code for each task
- ☐ Add execution-based filtering: run generated code on training pairs
- ☐ Add multi-representation prompting (at least 2–3 formats)
- ☐ Implement cell-level accuracy metric for graded reward
- ☐ Set up RL training loop (GRPO or PPO) with execution feedback

- [ ] Add test-time search (beam search → MCTS)
- [ ] Evaluate on ARC-AGI-1 public eval (400 tasks)